

Software Implementation and Testing Document

For

Group 1

Version 2.0

Authors:

Colby Leavitt

Dylan Connolly

Ivan Lapesii

Zack Lima

1. Programming Languages (5 points)

Python, JavaScript, and TypeScript. JavaScript and TypeScript were chosen for how widely used they are for the frontend. React comes with a lot of pre-made components which makes creating the frontend more efficient. Python/Django was chosen because a lot of the functionality that we need for the project comes with Django already which makes it easier to get started. Additionally, Django's interoperability with PostgreSQL facilitates a smoother integration process for routing from the backend to the frontend.

2. Platforms, APIs, Databases, and other technologies used (5 points)

As of right now, we haven't used too many technologies outside of React and Django. For our database, we are using PostgreSQL. We have also added simple-jwt and axios to help manage auth and api requests. React-datepicker was added to simplify user time slot selection. TailwindCSS has been our choice for frontend design.

3. Execution-based Functional Testing (10 points)

Student Functional Requirements

1. The system shall authenticate users with FSU credentials through either a confirmation to the student's e-mail, or via the system-wide Duo authentication provider from Cisco.
 - a. We tested our user authentication process by creating new student and admin-level users, then attempting to sign in both correctly and incorrectly. At this time, the application allows students to only login to the student page, while admin can login to the admin or student page. The application now correctly refreshes tokens on the front and backend, the testing was done by setting the access token time to 5s and ensuring correct refreshes on requests. Users not found in the database are unable to login or change any data, which we tested by trying to log in with an account that does not exist. We also updated the sign-in page to give more comprehensive feedback for failed login attempts.
2. The system shall display available study rooms in an interactive floor-map layout.
 - a. We have not implemented the floor-map layout yet.
3. The system shall allow filtering rooms by capacity, floor, library, and time availability.

- a. At this time, a user can display a library, its floors, and its rooms, and filter by the library. We tested the display functionality by running the application, and selecting different libraries, rooms, and floors to make sure the proper information was displayed. At this time, we have a room-first implementation and have not implemented sorting rooms by time availability.
4. The system shall display a user's upcoming reservations with room details and time information.
 - a. The system displays upcoming and past reservations, complete with room details and time information. It classifies bookings as confirmed, pending, or cancelled. We tested this by making and cancelling different types of reservations and observing the behavior of the page.
5. The system shall allow users to cancel their existing reservations.
 - a. Users are able to cancel confirmed and pending reservations. We tested this functionality by cancelling different types of reservations and making sure the results were displayed properly.
6. The system shall provide a timeline visualization of current and future room availability on a per-library and per-library-floor basis.
 - a. We have not implemented a timeline visualization of room availability.
7. The system shall enable search functionality for the library's loanable items.
 - a. Right now, the app displays a list of loanable items for each library, so there is not technically any search functionality yet. Display functionality has been implemented and tested on a baseline level by running the app and adding or removing loanable items from the database.
8. The system shall allow users to report issues with rooms or equipment.
 - a. Not implemented yet.
9. The system shall save user booking preferences for expedited future reservations.
 - a. Not implemented yet.
10. The system shall send confirmation emails for successful reservations.
 - a. Not implemented yet.
11. The system shall notify users when their reservation has been canceled by an administrator.
 - a. Not implemented yet.

Administrator Functional Requirements

1. The system shall allow administrators to mark individual rooms as unavailable due to maintenance.
 - a. This was tested for using manual test cases within the administrator panel. Once more comprehensive routing between frontend and backend is configured, we will write automated test cases.

2. The system shall display past, current, and upcoming room reservation records with student information.
 - a. This was also tested by using manual test cases of adding Room reservations onto a Library Floor. The past, current, and upcoming room reservations may be viewed at /admin/rooms/reservation/. Once our automated test suite is configured, we can perform more aggressive testing of this.
3. The system shall enable administrators to override or cancel existing bookings.
 - a. Similarly, we tested this manually by navigating the dialogs that may be seen at /admin/rooms/reservation/. By selecting the desired reservation to modify, administrators are redirected /admin/rooms/reservation/<reservation_pk_hash>/change/. Inside this page, administrators are free to modify the attributes of any reservation submitted by a student user.
4. The system shall provide functionality to update multiple room statuses simultaneously.
 - a. This has not been comprehensively tested as of yet, we have been able to perform bulk deletion operations of rooms from the /admin/rooms/room/ page, but not bulk modifications. This will have to be configured for administrative users.
5. The system shall enable the management of the library supply inventory.
 - a. This has been tested manually through the utilization of the frontend page and the backend administrator page, to view listed rentable items that belong to a specific library. We have not implemented individual item rentals or item quantities, but plan to as we move forward into the next increment.
6. The system shall allow tracking and responding to student-submitted issue reports.
 - a. This functional requirement has not been tested for, as it has not yet been implemented. As a lower-priority item, we are going to focus on it in the following two increments.
7. The system shall provide adjustment capabilities for maximum reservation time limits, on a per-library, per-library-floor, and per-library-floor-room basis.
 - a. Similarly, this functional requirement has not been tested for as we need to implement the associated administrator dialogs to accomplish these tasks, in addition to slightly modifying our current Django models.
8. The system shall track equipment rental analytics and identify frequently unavailable items.
 - a. As a lower-priority item, we plan to implement and test for this in the coming increments. As we move forward, we will develop the administrator dialogs necessary to fulfill this functionality.
9. The system shall provide booking analytics for space optimization and staffing decisions.
 - a. As a lower-priority item, we plan to implement and test for this in the coming increments. As we move forward, we will develop the administrator dialogs necessary to fulfill this functionality.

4. Execution-based Non-Functional Testing (10 points)

1. **Usability:** The system shall provide an intuitive interface requiring no training for students to make reservations.
 - a. We tested this by simply interacting with the UI to make sure that it was intuitive to use.
2. **Performance:** The system shall load floor maps and room availability data in no more than three seconds.
 - a. We tested this by running the app. At this point, we have not implemented floor maps, but all library data loaded in well under 3 seconds.
3. **Responsiveness:** The system shall function properly on mobile devices, tablets, and desktop computers.
 - a. We tested for this requirement by attempting to access our server from various devices we own, including our mobile phones, desktops, and tablets. In the next increment, we will perform a more comprehensive set of tests.
4. **Security:** The system shall protect user data and prevent unauthorized access to administrative functions.
 - a. We tested for this by attempting to access both administrative use-case functionality and student/grad student user use-case functionality from an external, unauthenticated endpoint. In both cases, access was denied to administrative functions, and the viewing of Libraries, Floors, Rooms, and Materials.
5. **Reliability:** The system shall prevent double-booking of rooms for any overlapping time intervals, and shall gracefully resolve simultaneous identical room reservation requests.
 - a. We tested this requirement by attempting to book the same room during the same time-slot. We have not yet tested for simultaneous booking attempt resolution, but our checks should prevent any issues. We will test this area more in the next increment.
6. **Accessibility:** The system shall comply with WCAG 2.1 AA standards for web accessibility.
 - a. We realized that these standards are too strenuous for our application at this time. We are in the process of finding new web accessibility standards to follow.
7. **Maintainability:** The system shall employ a modular architecture to facilitate future enhancements and maintenance.
 - a. We tested this by implementing new features modularly. For example, we were able to implement a materials class modeled after the previously existing classes

without making significant changes to the system framework. This is still the case in increment 2, where we were able to make modular modifications to the reservation architecture as well.

- 8. Availability:** The system shall be available 24/7 with scheduled maintenance windows not exceeding 4 hours per month.
 - a. Due to the current state of development, this non-functional requirement could not reasonably be tested for. However, we will maintain extended web server functionality across the next increment, and monitor the overall availability of the system, and total downtime.
- 9. Compatibility:** The system shall work correctly in modern web browsers (Chrome, Firefox, Safari, Edge, etc).
 - a. We have tested this by demoing the application on various web browser providers.
- 10. Data Integrity:** The system shall maintain accurate reservation data even during concurrent booking attempts.
 - a. The system maintains accurate reservation data, but we still need to protect against data corruption/confusion during concurrent booking attempts. However we are able to make sure that a room is not booked multiple times during the same time slot.

5. Non-Execution-based Testing (10 points)

To perform non-execution-based testing, we utilized git to review each other's pull requests before merging them with our first-iteration branch. We also communicated via Discord to walk each other through new features and challenges, as well as allow each other to review each other's contributions. Discord communications included live video meetings, recorded videos, and simple messages.